

Practicum No.9**Date:.....****Stack Operations Using Registers****I. Practical Significance**

Modeling stack operations deepens understanding of how PUSH and POP manage memory during program execution. Learners explore stack behavior, pointer updates, and subroutine handling through register-based simulations. Grasping stack logic is critical for recursion, interrupt service routines, and function call mechanisms. These concepts support effective memory and process control in low-level programming

II. Competency and Industry-Specific Practical Skills

This practicum exercise is expected to develop the following skills for the industry-identified competency.

- **Use** scalable architectures for intelligent systems to ensure reliability.

(As stated in the Course Structure)

- a) **Create** stack setup for handling function calls and returns. *(PDO)*

III. Relevant CO(s) *(As stated in the Course Structure)*

- CO1. **Interpret** the system functionality of a computer architecture.

IV. Practical Outcome (PrO) *(As stated in the Course Structure)*

- **Simulate** stack PUSH and POP logic with **the given** registers using MATLAB.

V. Related ADO(s) *(As stated in the Course Structure)*

- **Follow** industry standards for system evaluation and performance optimization.
- **Function** effectively in teams to develop processor and memory architectures.
- **Function** responsibly in leadership roles to design computing systems.

VI. Minimum Underpinning Theory *(Include relevant content & images for this practicum.)*

Understanding stack operations is essential for managing function calls, interrupts, and local data storage. A stack operates on the Last-In-First-Out (LIFO) principle, where data is “pushed” onto the top of the stack and later “popped” in reverse order. The stack is typically managed using a stack pointer (SP) register, which keeps track of the current top of the stack in memory. The PUSH instruction decreases the stack pointer and stores data, while the POP instruction retrieves data and increases the stack pointer.

Students should learn how the PUSH and POP operations affect the register and memory content, especially during nested subroutine calls or recursive functions. They should understand the impact of these operations on the flag register and CPU state, and how register pairs or individual registers are preserved during execution. Instruction formats, memory address allocation, and the role of stack segments must be studied. Using simulation tools, students can observe the behavior of registers during stack operations and design logical sequences to implement these functions.

VII. Algorithm *(Include probable photos of experimental set-up.)*

1. **Start** the program.
2. **Initialize** the stack pointer $SP = 0$.
3. **Display** message “Register Stack”.
4. **Perform** PUSH operation:
5. **Increase** SP by 1 and display value.
6. **Increase** SP by 1 again and display value.
7. **Perform** POP operation:
8. **Decrease** SP by 1 and display value.
9. **End** the program.

Key Concepts for Students:

- In register-based stacks, the pointer usually *increases* when pushing data.
- In memory-based stacks, the pointer usually *decreases* when pushing (stack grows downward in memory).
- POP reverses the direction of PUSH (subtract for register stack, add for memory stack).

▼ Workspace

Name	Value	Size	Class
SP	1	1×1	double

VIII. Resources Required

S. No.	Resource	Suggested Broad Specification	Quantity
1	Laptop	Intel Core i5/i7 processor, 8GB RAM, Windows OS recommended for 8086 emulator	1 No.
2	8086 Emulator	EMU8086 Microprocessor Emulator https://emu8086.en.lo4d.com/windows	1 No.
3	MATLAB	https://matlab.mathworks.com/	1 No.
4	Turbo Assembler	GUI Turbo Assembler (TASM) https://sourceforge.net/projects/guitasm8086/	1 No.

IX. Safety Precautions

- Do not run unfamiliar or unverified scripts found online which may harm your computer.
- Verify your register usage and memory addresses to avoid system-level crashes or infinite loops.
- Close all other programs before executing large or unknown assembly code to free up resources.
- Use licensed version of MATLAB to avoid security or legal issues.

X. Procedure (Self-Instructional)

- Switch on the computer.
- Open MATLAB on your computer.
- Create a new script file (File → New → Script).
- Run the following MATLAB script and note the values on the console.

```
% register_stack_simple.m
clc; clear;
```

```
SP = 0; % stack pointer starts at 0
disp('--- Register Stack ---');
```

```
% PUSH
SP = SP + 1; fprintf('PUSH -> SP = %d\n', SP);
SP = SP + 1; fprintf('PUSH -> SP = %d\n', SP);
% POP
SP = SP - 1; fprintf('POP -> SP = %d\n', SP)
```

```
--- Register Stack ---
PUSH -> SP = 1
PUSH -> SP = 2
POP -> SP = 1
```

▼ Workspace

Name	Value	Size	Class
SP	1	1×1	double

- Shut down the computer.

XI. Actual Resources Used

S. No.	Name of Resource	Broad Specifications		Data	Remarks (If any)
		Tool	Version		
1.	Laptop	MATLAB	R2023a	SP initialized to 0	Smooth execution
2.	Operating System	Windows	11	RAM = 8GB	No issues
3.	Emulator	EMU8086	v4.08	Stack simulation	Optional use
4.	Assembler	TASM	v5.0	Register operations	Verified logic

XII. Actual Procedure Followed

1. Opened MATLAB and created a new script file for stack simulation.
2. Initialized stack pointer (SP = 0) and executed PUSH and POP operations.
3. Observed and recorded changes in stack pointer values in the command window.

XIII. Observations and Dimensional Measurement

S. No.	Observations	Data	Remarks
1	Initial Stack Pointer	SP = 0	Starting point
2	After First PUSH	SP = 1	Incremented
3	After Second PUSH	SP = 2	Incremented
4	After POP	SP = 1	Decrement

XIV. Results/Observations

1. PUSH operation increases the stack pointer value, indicating addition of data to the stack.
2. POP operation decreases the stack pointer, showing removal of data.
3. The stack follows Last In First Out (LIFO) behavior.

XV. Interpretation of Results

The experiment demonstrates that stack operations are controlled using the stack pointer. PUSH operations increment the pointer, while POP operations decrement it. This confirms the LIFO principle, where the last inserted element is removed first.

The stack structure ensures proper handling of function calls and temporary data storage, making it essential in program execution and memory management.

XVI. Conclusions

The stack operates based on the LIFO principle and is efficiently managed using the stack pointer. PUSH and POP operations correctly modify the pointer value, ensuring proper data handling. This concept is crucial for function calls, recursion, and interrupt handling in computer systems.

XVII. Suggested Practicum Related Questions:

Note: Below are a few questions related to industry-specific higher-order thinking skills (HOT) that teachers must use to ensure students achieve the predetermined course outcomes.

1. **Explain** how using a stack helps keep track of values during a program.
2. **Evaluate** whether stack or registers are better for saving data, and justify your reasoning.
3. **Modify** the PUSH/POP logic to take in three numbers and return them in reverse order.
4. **Predict** what will happen if more data is PUSHed than the stack can hold.
5. **Justify** why the stack follows a Last In First Out (LIFO) rule.
6. **Describe** how POP identifies which data to remove first.
7. **Assess** why it is important to balance PUSH and POP operations.
8. **Predict** the outcome if a POP is attempted on an empty stack.
9. **Demonstrate** how a stack stores return addresses during function calls.
10. **Explain** why the stack is often referred to as “temporary memory.”

XVIII. References / Suggestions for further reading

- a) Data Transfer Instructions in 8086 Microprocessor: <https://www.geeksforgeeks.org/computer-organization-architecture/data-transfer-instructions-8086-microprocessor/>
- b) Assembly Registers: https://www.tutorialspoint.com/assembly_programming/assembly_registers.htm
- c) Types of Registers in 8086 Microprocessor: <https://www.geeksforgeeks.org/electronics-engineering/types-of-registers-in-8086-microprocessor/>
- d) 8086 Microprocessor Architecture: <https://microcontrollerslab.com/8086-microprocessor-architecture/>
- e) Stack Introduction – PUSH & POP - [YouTube](#) – Education 4u
- f) Stack Organization – Register and Memory Stack – [YouTube](#) – Education 4u
- g) RISC and CISC Architecture - <https://intellipaat.com/blog/difference-between-risc-and-cisc/>

XIX. Suggested Assessment Scheme

The performance indicators given serve as a guideline for assessing the ‘*process-related skills/LOs*’ (marks to be awarded in real-time in the laboratory by the faculty member, as these cannot be measured after the practicum is over) and ‘*product-related skills/LOs*’

Note: The weightage for the **process-related skills** and **product-related skills** will change depending on the PrO, COs, and the competency related to the concerned course.

Performance Indicators		Maximum Marks	Marks Obtained
Process-related Skills: 18 Marks - 60%			
1	<u>Follow</u> industry standards	3	
2	<u>Function</u> effectively in teams	2	
3	<u>Function</u> responsibly in leadership	3	
4	<u>Create</u> stack setup for handling function calls and returns.	10	
Product-related Skills: 12 Marks - 40%			
5	Uses correct commands or syntax in Tool	3	
6	Results	3	
7	Interpretation of Results	3	
8	Conclusion	3	
Total		30	

To be filled by Student:

S.No.	Name of the Student	Register No.
1	Neavis Rishva. J	URK25CS1236

To be filled by the Faculty Member:

Marks Obtained	Name and Designation of the Faculty Member	Date of Evaluation
----------------	--	--------------------

Process-Related Skills (18)	Product-Related Skills (12)	Total (30)		

